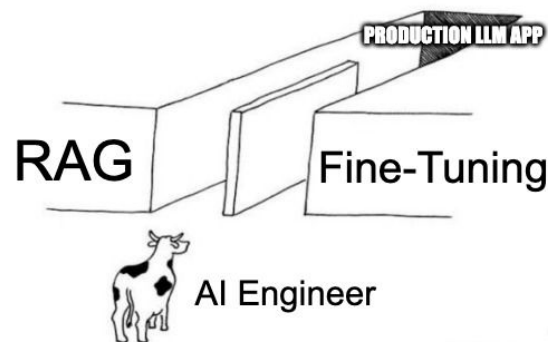
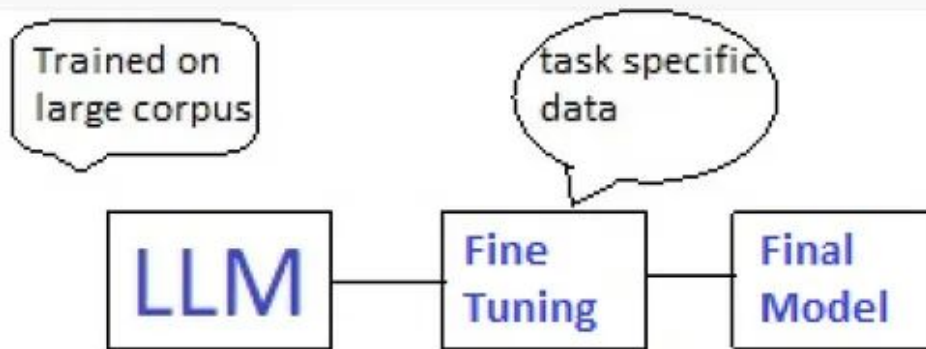


LoRA for Parameter Efficient Fine Tuning: Implementation & Evaluation on Consumer GPUs

Adib Akkari, Omar El Masaoudi, Tanim Chowdhury

The LLM Fine-Tuning Bottleneck

- Full fine-tuning = update all parameters
- Billion-scale LLMs require 100–400GB+ VRAM
- Training takes days–weeks on enterprise GPUs
- Most researchers cannot access such compute
- Thus, there is a need for more accessible fine-tuning method





Solution? Parameter-Efficient Fine-Tuning (PEFT)

What is PEFT?

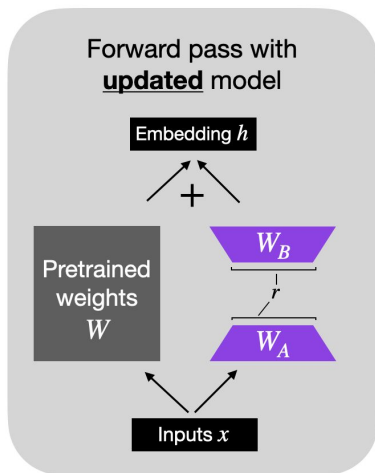
- Freeze base model weights (don't update them)
- Train only small adapter modules (~1% of parameters)
- Achieve comparable performance with 99%+ parameter reduction

Why PEFT?

- Fast Training: 10-100x faster than full fine-tuning
- Accessible: Works on consumer GPUs (RTX 3090, RTX 4090)
- Memory Efficient: Gradients & optimizer states only for adapters

PEFT algorithms: LoRA

LoRA weights, W_A and W_B , represent ΔW



**Training an LLM
from scratch**

**Fine-tune with
QLoRA**



What is Low Rank Adaptation (LoRA)?

- Instead of updating W , learn $\Delta W = A \cdot B$ (rank- r)
- Insert small trainable matrices into attention layers
- Base model stays frozen
- Achieves strong performance with 0.24–0.47% trainable parameters (GPT-Neo, GPT-2)

Design:

- Matrix A: Gaussian initialization (mean=0, std=0.02)
- Matrix B: Initialized to zeros
- Rank ' r ' < dimensions (hence the name low rank)

We approximate $\Delta W = A \cdot B$

Rank Decomposition Matrix


$$W_{\text{ft}} = W_{\text{pt}} + \underbrace{\Delta W}_{\text{Approximation}} = W_{\text{pt}} + \underbrace{AB}_{\text{Rank Decomposition Matrix}}$$

where $W_{\text{ft}}, W_{\text{pt}}, \Delta W, AB \in \mathbb{R}^{d \times d}$

and $\underbrace{A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}}_{\text{Low Rank}}$

$$\text{weight}_{\text{new}} = W + \frac{\alpha}{r} \cdot (W_A \cdot W_B)$$



Experiment

We utilized Consumer level Hardware

Hardware:

- NVIDIA RTX 3090 — 24GB VRAM
 - Which is good for loading and updating model params FAST !!
- Intel ultra 7 265 kf
- 32 GB DDR5 RAM

Approach:

- Implemented LoRA from scratch
- Test on Language Modelling, QA datasets
- Attempted CorDA initialization (SVD-based), but unstable on small models.



Datasets

- **Wikitext-2** — language modeling (36.7K)
- **IMDB** — long-form sentiment (25K)
- **SST-2** — short-form sentiment (67.3K)
- **SQuAD v2** — extractive QA (130.3K) → Used to test generalization across diverse tasks.



HyperParameters & Model Architecture

Learning rate: $1e-4$

Batch size: 8 + gradient accumulation 2 (eff. batch ~16)

Steps: 4000 (10-15 minutes on RTX 3090)

Warmup: 10% of steps

Model	Total Parameter	LoRA trainable parameters	%
GPT-Neo-125M	125M	589,824	0.47%
GPT-2	124M	295,000	0.24%



Reality Check

- Initial target: **Qwen 2.5 (4B parameters)**

Results:

- Load time: **45+ minutes**
- VRAM overflow even with 4-bit quantization → Not feasible on 24GB GPU (Practical bottleneck)
- Pivot to **GPT-Neo (128M parameter) and GPT-2 124M instead**



Key Results

DataSet	Baseline	LoRA	Change	Result
Wikitext Perplexity	54.26	31.64	-47%	worked
SST-2 Accuracy	0.44	0.56	+27.3	worked
IMDB Accuracy	0.26	0.59	-14%	failed
SQuAD F1	0.130	0.121	-7%	failed



Cont.

Strong on language modeling (41.7% improvement!)

Mixed on sentiment (SST-2 up, IMDB down)

Weak on QA (architectural mismatch for span extraction)



Lessons Learned

What Worked:

- Parameter Efficiency (PEFT) validated (0.24-0.47% trainable parameters)
- Strong LM improvements with small models ~ 27.1 +% on short-form sentiment classification tasks.
- Training feasible on consumer GPU (10-15 minutes, <24GB VRAM).
- From-scratch LoRA implementation behaved as intended compared to the hugging face implementation as a baseline.

Trade-offs & Challenges

- Hyperparameters highly sensitive: short schedule failed, long schedule worked
- Single-task LM training harms sentiment/QA
- CorDA SVD initialization unstable and did not converge on small models
- Hardware bottlenecks (model loading, activation memory) matter more than theory suggests



Conclusion

- PEFT LoRA democratizes LLM research - works on consumer hardware
- Makes adaptation possible on consumer GPUs
- Hyperparameter tuning is critical for stability → conservative settings beat aggressive ones
- Hardware realities (VRAM, load time) must be acknowledged - loading times are real bottlenecks
- Small models still benefit significantly from PEFT
- Implementation transparency helps - from-scratch code validates theory
- Provides a reproducible path for researchers without enterprise compute
- Enables researchers without enterprise GPUs to adapt LLMs
- Documents practical constraints literature often ignores